RSA vs PostQuantum Cryptography

By Ademola Baruwa

N1142033

ABSTRACT

The rapid advancement of quantum computing threatens the foundational security of RSA, which relies on the intractability of large-integer factorization. This dissertation presents a side-by-side comparison of classical RSA (paired with AES for bulk encryption) and the lattice-based post-quantum scheme CRYSTALS-Kyber under Shor's algorithm assumptions. Both systems were implemented in Python—RSA+AES using PyCryptodome and CRYSTALS-Kyber via the PQClean library—and benchmarked across file sizes ranging from 500 bytes to 500 KB. Key-generation, encapsulation/encryption, and decryption times were recorded over 100 trials, alongside CPU and memory usage. Results show that CRYSTALS-Kyber key generation is three orders of magnitude faster than RSA (≈3 ms vs. ≈0.9 s for 2048-bit keys), and encapsulation/decryption operations consistently outperform RSA's asymmetric steps, while AES throughput remains under 3 ms for large payloads. These findings demonstrate that post-quantum KEMs can be integrated into existing hybrid frameworks with negligible performance penalty, offering quantum resistance without sacrificing practicality. Future work will explore hardware acceleration (e.g., GPU or FPGA), protocol-level integration in TLS, and evaluations of other NIST-candidate schemes to guide a seamless migration toward quantum-secure communications.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my tutor, Dr Amir Pourabdollah for his patient guidance, thoughtful feedback, and unwavering support throughout this project.

I also owe my heartfelt thanks to my family for their constant understanding and encouragement. Their belief in me, especially during the late nights of coding and writing, provided the confidence and motivation I needed to see this work through to completion.

TABLE OF CONTENTS

CONTENTS

s

LIST OF FIGURES AND TABLES

CHAPTER ONE : INTRODUCTION

Quantum algorithms such as Shor's Algorithm can factor large integers in polynomial time, directly challenging the security of RSA, which relies on the presumed intractability of integer factorization for sufficiently large moduli. This vulnerability gives rise to the "store-now, decrypt-later" threat, whereby data encrypted today may be compromised once quantum-capable adversaries emerge. To safeguard long-term confidentiality, it is essential to evaluate quantum-resistant alternatives.

A BRIEF OVERVIEW OF RSA

RSA (Rivest–Shamir–Adleman, 1978) is a public-key cryptosystem that generates a key pair (e,n)(e,n)(e,n) for encryption and (d,n)(d,n)(d,n) for decryption, where n=p×qn = p \times qn=p×q is the product of two large primes. Under classical assumptions, factoring a 2048-bit modulus nnn into its prime factors ppp and qqq is computationally infeasible. Consequently, RSA underpins secure web protocols (HTTPS), virtual private networks, and digital signatures. However, if integer factorization becomes efficient on quantum hardware, RSA's foundational security premise collapses.

A BRIEF OVERVIEW OF SHOR'S ALGORITHM

Shor's Algorithm (Shor, 1994) exploits quantum superposition and the Quantum Fourier Transform to factor integers and compute discrete logarithms in polynomial time. On a sufficiently fault-tolerant quantum computer, it can recover RSA private keys by factoring the modulus nnn in O((log⁡n)3)O((\log n)^3)O((logn)3) operations—far faster than the best classical methods. Although today's Noisy Intermediate-Scale Quantum (NISQ) devices remain far from this capability, rapid progress in qubit counts and error-correction techniques suggests that practical quantum attacks may be feasible within the next decade

AIMS AND OBJECTIVES

The aim of this dissertation is to compare the security guarantees and performance overheads of classical RSA (with AES for bulk encryption) against the lattice-based post-quantum Key Encapsulation Mechanism (KEM) CRYSTALS-Kyber. This comparison will inform recommendations for integrating quantum-resistant algorithms into existing protocols.

AIM

Evaluate and contrast RSA+AES and CRYSTALS-Kyber in terms of security properties and practical performance.

OBJECTIVES

- Implement and benchmark RSA+AES (2048-bit key).
- Integrate CRYSTALS-Kyber KEM at security levels Kyber512, Kyber768, and Kyber1024.
- Measure performance metrics (key generation, encapsulation/encryption, decryption) over 100 trials for payloads of 500 B, 1 KB, 10 KB, and 500 KB.
- Profile CPU and memory usage of all cryptographic operations using psutil.
- Analyze trade-offs and propose guidelines for migrating to post-quantum schemes.

By systematically addressing these objectives, this study provides empirical data on the feasibility of adopting quantum-resistant cryptography without compromising performance.

CHAPTER TWO: CONTEXT & LITERATURE REVIEW

This chapter examines current cryptographic methods, focusing on classical RSA encryption, quantum computing, and emerging post-quantum cryptography. This review aims to provide a comprehensive knowledge of the underlying technologies in use today, define the state-of-the-art in each field, and identify important constraints and problems for developing more robust security solutions in the quantum age.

For decades, RSA encryption has served as a cornerstone of secure digital communication. Its security is based on the computational complexity of factoring huge integers—a problem that, under traditional computer paradigms, is deemed practically impossible. However, as quantum computing advances, RSA becomes increasingly vulnerable. The concept of "store now, decrypt later" exemplifies this risk, as data captured now may be decrypted in the future when quantum computers capable of performing algorithms like Shor's become available.

Along with the study of RSA, this chapter covers the fundamental notions of quantum computing. Quantum computing uses phenomena like superposition and entanglement to accomplish calculations in ways that traditional computers cannot. This article is centered on Shor's method, a quantum method that demonstrates the theoretical feasibility of effectively

factoring enormous numbers in polynomial time. This capacity immediately undermines the basic security assumption of RSA encryption.

The chapter explores post-quantum encryption, which uses secure methods to withstand attacks from quantum computers. This article introduces post-quantum algorithms such as lattice-based, code-based, hash-based, and multivariate cryptography. It also discusses their present performance issues and limitations.

The chapter provides a contextual framework for further examinations, including practical demonstrations of RSA and Shor's Algorithm, as well as a comparison of classical and post-quantum cryptography systems.

2.2 CLASSICAL CRYPTOGRAPHY: RSA IN PRACTICE

2.2.1 OVERVIEW OF RSA.

RSA (Rivest–Shamir–Adleman, 1977) is a public-key cryptosystem that secures digital communications by generating a key pair (e,n)(e,n)(e,n) for encryption and (d,n)(d,n)(d,n) for decryption, where n=p×qn = p \times qn=p×q is the product of two large prime numbers. Under classical computing assumptions, factoring a 2048-bit modulus nnn into its prime factors is computationally infeasible. Consequently, RSA underpins HTTPS, virtual private networks, and digital signatures, offering data confidentiality and integrity.

2.2.2 LIMITATIONS OF RSA.

Despite its pervasive adoption, RSA exhibits several drawbacks:

- **Quantum Vulnerability:** Quantum algorithms—most notably Shor’s Algorithm—can factor large integers in polynomial time, rendering RSA insecure against sufficiently capable quantum adversaries.
- **Performance Overhead:** Increasing key sizes (e.g., from 2048 to 4096 bits) to counter emerging threats incurs significantly higher computational cost and latency.
- **Key Management Challenges:** Secure generation, storage, and rotation of large RSA key pairs introduce operational complexity, especially in resource-constrained environments.

2.2.3 Demonstration of RSA Operation

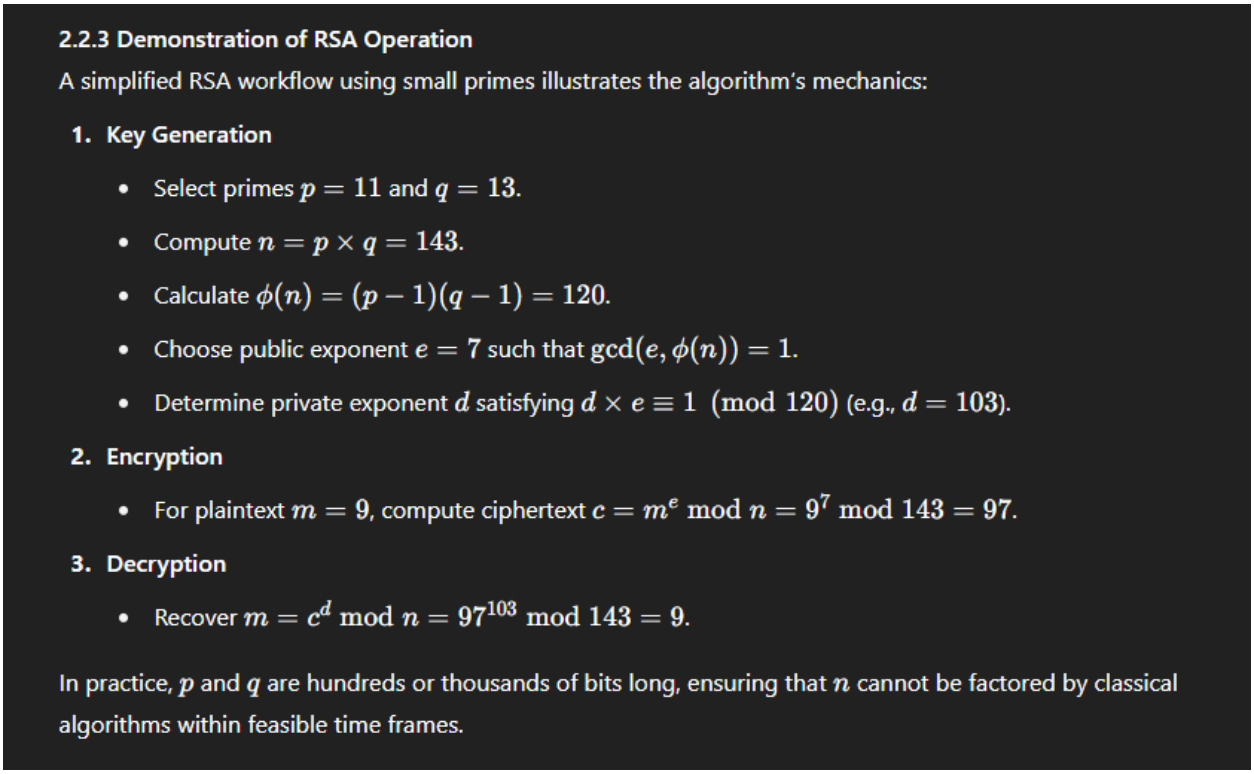


Figure 1 above shows the RSA process, from key generation to encryption and decoding. In reality, the primes ppp and qqq are selected to be exceedingly large (hundreds or thousands of bits) so that the modulus nnn cannot be factorized by any classical algorithm. The example also emphasizes RSA's central security assumption: the difficulty of factoring nnn into its constituent primes.

2.3 Quantum Computing and the Shor Algorithm

2.3.1 Quantum Computing Essentials

Quantum computing is a fundamentally novel approach to information processing that takes advantage of quantum mechanics' unique characteristics like superposition and entanglement. In contrast to classical bits, which can either be 0 or 1, quantum bits (qubits) can be in both states at the same time. This characteristic enables quantum computers to process multiple potential solutions concurrently, allowing them to solve specific problems tenfold faster than classical computers (Nielsen & Chuang, 2010).

State of the Art:

IBM, Google, and IonQ have driven progress in quantum processor scale and fidelity. IBM's roadmap evolved from 5-qubit systems in 2016 to 127-qubit devices by 2021 (IBM Quantum, 2020) while Google's 53-qubit "Sycamore" processor achieved a landmark demonstration of quantum supremacy in 2019 . IonQ's trapped-ion platforms likewise exceed 100 qubits, offering

high-fidelity gate operations. These Noisy Intermediate-Scale Quantum (NISQ) systems support proof-of-concept implementations of algorithms—such as variational quantum eigensolvers and small-scale Shor's runs—despite noise and limited coherence. Achieving cryptographically relevant computations (e.g., factoring 2048-bit integers) will demand fault-tolerant architectures with scalable quantum error correction, a milestone expected only once physical qubit counts and error thresholds meet stringent thresholds (Preskill, 2018)

Limitations:

Despite significant progress, current NISQ systems face three principal challenges that limit their applicability to cryptographic tasks. First, environmental noise induces qubit decoherence, causing quantum states to degrade within microseconds and reducing available computation time (Preskill, 2018) . Second, gate fidelity remains below fault-tolerance thresholds: typical two-qubit gate error rates range from 0.1% to 1%, leading to cumulative errors in deeper circuits (Devitt et al., 2013) . Third, implementing quantum error-correction to suppress these errors requires orders of magnitude more physical qubits per logical qubit, drastically increasing hardware overhead and complicating system scaling. As a result, although NISQ devices can execute small-scale demonstrations of algorithms—including toy instances of Shor's factoring—they fall far short of the millions of logical qubits needed to factor a 2048-bit RSA modulus reliably.

2.3.2 SHOR'S ALGORITHM OVERVIEW:

Shor's Algorithm (Shor, 1994) factors an nnn-bit integer in polynomial time by reducing the problem to period finding, which is efficiently solved via the Quantum Fourier Transform. The algorithm comprises three phases:

1. **Superposition Preparation:** Initialize quantum registers into a uniform superposition of states.
2. **Modular Exponentiation & QFT:** Apply a modular exponentiation oracle followed by the Quantum Fourier Transform to extract the period rrr of the function f(x)=axmodNf(x)=a^x \bmod Nf(x)=axmodN.
3. **Classical Post-Processing:** Use the measured period rrr to compute non-trivial factors of NNN by evaluating gcd⁡(ar/2±1,N)\gcd(a^{r/2} \pm 1, N)gcd(ar/2±1,N).

This approach runs in O((log⁡N)3)O((\log N)^3)O((logN)3) time, a dramatic improvement over the best classical factorization algorithms, which require super-polynomial time for large NNN

Cutting-edge implementations and demonstrations:

Proof-of-concept implementations of Shor's Algorithm have been executed on both quantum hardware and simulators. Martín-López et al. (2012) demonstrated factoring N=15N=15N=15

and N=21N=21N=21 using photonic qubits, validating the core algorithmic steps despite severe resource constraints . Subsequent experiments have incrementally increased NNN, but remain orders of magnitude below cryptographically relevant sizes.

IMPLICATIONS OF RSA:

Translating Shor's theoretical efficiency into practice faces three major hurdles:

Qubit Count & Fidelity: Factoring a 2048-bit modulus requires millions of error-corrected logical qubits; current devices provide only tens to hundreds of noisy qubits (Gidney & Ekerå, 2019)

Error Correction Overhead: Implementing fault-tolerant gates demands substantial physical-to-logical qubit ratios, inflating hardware requirements by two to three orders of magnitude.

Circuit Depth & Decoherence: Deep circuits needed for modular exponentiation accumulate errors faster than current coherence times allow.

Until these challenges are resolved—through advances in qubit technology, error-correction protocols, and system integration—Shor's Algorithm remains a powerful theoretical threat, but not yet a practical one for breaking standard RSA key sizes.

2.4 POSTQUANTUM CRYPTOGRAPHY

2.4.1 OVERVIEW OF PQC.

Post-quantum cryptography (PQC) encompasses algorithms designed to remain secure against both classical and quantum adversaries. Unlike RSA or ECC—which rely on the hardness of integer factorization or discrete logarithms—PQC schemes are founded on problems such as lattice-based Learning With Errors (LWE), code-based decoding, and multivariate polynomial systems, for which no efficient quantum algorithms are known . In response to the looming quantum threat, NIST has led a multi-year standardization effort, evaluating lattice-, code-, hash-, and multivariate-based candidates to establish future-proof cryptographic standards

2.4.2 MAJOR FAMILIES AND THEIR LIMITATIONS

Post-quantum cryptographic algorithms are roughly classified into various families, each with its own strengths and weaknesses:

 Lattice-Based Cryptography (e.g., CRYSTALS-Kyber, FrodoKEM): Security reduces to worst-case lattice problems and LWE, believed hard for quantum computers. Drawbacks include larger public keys and ciphertexts compared to RSA (Regev, 2006)

 Code-Based Cryptography (e.g., McEliece, Niederreiter): Based on the hardness of decoding random linear codes; offers fast operations but public keys often exceed hundreds of kilobytes (McEliece, 1978)

 Hash-Based Signatures (e.g., SPHINCS+): Rely solely on well-studied hash functions; signatures are large and stateful use can complicate key management (Bernstein & Lange, 2017)

 Multivariate Cryptography (e.g., UOV, Rainbow): Grounded in solving multivariate quadratic equations; offers compact keys but several schemes have been broken, raising durability concerns (Ding & Petzoldt, 2017)

2.4.3 Implementation and Performance Challenges

While post-quantum cryptographic algorithms offer increased security against quantum assaults, their practical implementation presents various challenges:

PQC methods often have bigger key sizes and slower encryption and decryption speeds than standard systems like RSA. This can result in additional computational and storage overhead, thus limiting their utility in resource-constrained contexts.

These algorithms are resistant to all known quantum assaults. However, new quantum or classical techniques may still arise. As a result, continual research and thorough analysis are required to ensure that these plans stay viable in the long run.

Transitioning from traditional cryptographic approaches to PQC will require considerable changes in software and hardware infrastructures. Interoperability concerns may occur during the transition period, and widespread adoption will be dependent on reaching an agreement on standardization. The current standardization initiatives by organizations such as NIST are an important step in this process, but full acceptance across all industries is likely to take time (NIST, 2016).

2.5 Limitations of Current Solutions and Research Gaps

Despite RSA encryption's long-standing success in securing digital communications, certain fundamental limitations and research gaps persist, emphasizing the necessity for more inquiry and the development of alternative solutions.

RSA-Specific Gaps:

RSA's security relies on the classical difficulty of factoring large moduli; however, this premise will collapse under sufficiently powerful quantum attacks (Shor, 1994) Increasing key sizes (e.g., from 2048 to 4096 bits) can delay classical threats but incurs exponential computational and storage costs, impeding performance and scalability. Moreover, managing large RSA key-pairs—secure generation, distribution, rotation—adds operational complexity, particularly for resource-constrained devices and large deployments.

Post-Quantum Cryptography Challenges:

Post-quantum algorithms promise resilience against both classical and quantum adversaries, yet they introduce new hurdles:

- **Maturity and Cryptanalysis:** Many candidates remain experimental, lacking extensive cryptanalytic vetting and side-channel hardening (NIST, 2016)
- **Parameter Overhead:** Larger key and ciphertext sizes inflate bandwidth and storage requirements, complicating adoption in embedded systems.
- **Integration Effort:** Extensive modifications to existing protocols (e.g., TLS), libraries, and hardware accelerators are required before widespread deployment.

Need for comparative studies:

Despite theoretical analyses, **practical**, code-level benchmarks comparing RSA and PQC across realistic payloads and platforms are scarce. In the absence of such studies, organizations cannot accurately quantify trade-offs in latency, CPU/memory usage, and implementation effort. This lack of user-friendly, reproducible performance data—especially for IoT and SME contexts—represents a significant research gap.

2.6 SUMMARY.

In conclusion, while RSA has long been the dominant approach for protecting digital communications, its reliance on the difficulties of integer factorization makes it vulnerable to future quantum attacks. Shor's Algorithm, with its capacity to factor big numbers in polynomial time, poses a significant danger to RSA by potentially compromising its security foundation once quantum hardware advances sufficiently. Post-quantum cryptography is a viable solution to withstand quantum attacks. However, obstacles include higher key sizes, slower computational efficiency, and inadequate standardization.

CHAPTER 3: NEW METHOD AND PROOF-OF-CONCEPT IMPLEMENTATION

This chapter introduces a novel method designed to address the limitations identified in Chapter 2, focusing on the development of an optimized post-quantum encryption system using CRYSTALS-Kyber. The chapter describes a proof-of-concept implementation that not only demonstrates classical RSA encryption but also integrates CRYSTALS-Kyber encryption and a toy quantum factorization demonstration using Qiskit. By outlining the theoretical foundations, design components, implementation steps, and evaluation strategies, this chapter provides a comprehensive roadmap for the proposed solution and sets the stage for further technical implementation details in subsequent chapters.

3.1 INTRODUCTION.

Traditional public-key systems like RSA face decisive threats from quantum algorithms such as Shor's Algorithm, which factors large integers in polynomial time (Shor, 1994) . In contrast, lattice-based schemes—exemplified by CRYSTALS-Kyber—rely on the hardness of the Learning With Errors (LWE) problem and are believed resistant to both classical and quantum attacks (Regev, 2006) . This chapter outlines a side-by-side implementation of RSA+AES (using PyCryptodome) and CRYSTALS-Kyber (via PQClean), alongside a Qiskit-based toy demonstration of quantum factorization. Section 3.2 reviews relevant background; Section 3.3 justifies the new approach; Section 3.4 describes OPQCF's architecture and pseudocode; Section 3.5 highlights expected benefits; Section 3.6 discusses limitations; and Section 3.7 previews Chapter 4's implementation details.

3.2 BACKGROUND.

3.2.1 RSA vulnerabilities and classical cryptography

RSA's security hinges on the classical intractability of factoring a large modulus n=p×qn=p\times qn=p×q (Rivest et al., 1978). However, Shor's Algorithm reduces integer factorization to polynomial time on a fault-tolerant quantum computer, rendering RSA insecure against future quantum adversaries (Shor, 1994; Mosca, 2018)

3.2.2 Quantum Threats & Computational Principles

Quantum computing uses the characteristics of superposition and entanglement to conduct operations on many states at once, allowing quantum algorithms to solve problems significantly more effectively than classical algorithms (Nielsen & Chuang, 2010). Shor's Algorithm, in particular, can significantly reduce the time necessary for integer factorization, posing a danger to RSA security. Current quantum devices, such as Noisy Intermediate-Scale Quantum (NISQ) systems, are not yet capable of breaking RSA at realistic key sizes. However, advancements in qubit counts and error correction techniques indicate that this threat is imminent (Mosca, 2018).

3.2.3 Theoretical Foundations of Crystals and Kyber.

CRYSTALS-Kyber is a KEM built on the hardness of Ring-LWE, offering strong security reductions under worst-case lattice problems and efficient key-generation and encapsulation operations (Bos et al., 2021; NIST, 2022) . Optimizing Kyber for practical use involves tuning error distributions and polynomial arithmetic to minimize ciphertext sizes and computational overhead.

3.3 JUSTIFICATION OF THE NEW IDEA

3.3.1 Addressing the Need for Post-Quantum Solutions

The reason for building a new approach stems from two difficulties mentioned in Chapter 2: RSA's vulnerability to quantum attacks and the practical constraints of existing post-quantum solutions. RSA's reliance on integer factorization difficulty makes it unsuitable for a future in which quantum computers may efficiently perform Shor's Algorithm (Shor, 1994). Although post-quantum approaches like CRYSTALS-Kyber provide quantum resistance, their application in real-world systems is often hampered by inefficiencies such as huge key sizes and high computing costs (NIST, 2016, 2022; Regev, 2006).

3.3.2 Selection of Crystals: Kyber

CRYSTALS-Kyber was chosen as the post-quantum approach for this project due to its strong theoretical foundations, active review in the NIST PQC process, and demonstrated potential for practical deployment (Bos et al., 2021). Kyber offers a good blend of security and performance, making it an excellent choice for a proof-of-concept implementation. The project compares RSA with CRYSTALS-Kyber to show the trade-offs between conventional and post-quantum cryptography systems.

3.3.3 Novelty of the Comparative Approach

This work is unique in that it compares two encryption systems, RSA and CRYSTALS/Kyber, This comprehensive approach fills a research gap: whereas many studies focus on theoretical elements of quantum resistance, few offer realistic code-level comparisons that evaluate performance parameters like encryption speed, resource utilization, and security margins. This study not only highlights the immediate impact of quantum weaknesses on RSA, but also gives

actual data on the benefits and limitations of CRYSTALS-Kyber. This provides vital insights for future cryptographic standardization and implementation.

3.4 Detailed Description of the Proposed Idea and Implementation

3.4.1 Conceptual Overview

The proposed system, termed the Optimized Post-Quantum Cryptographic Framework (OPQCF), integrates 2 core components as shown in Figure 2 below:

1. **Classical RSA Module:** Implements standard RSA encryption and decryption to serve as a baseline for comparison.
2. **CRYSTALS-Kyber Module:** Implements a lattice-based encryption scheme based on CRYSTALS-Kyber, optimized to reduce key sizes and computational overhead.

```
// RSA+AES Hybrid
function RSA_AES_Encrypt(data):
    (pk, sk) = RSA_KeyGen(2048)
    aes_key = RandomBytes(32)
    (nonce, ciphertext, tag) = AES_Encrypt(data, aes_key)
    wrapped_key = RSA_Encrypt(aes_key, pk)
    return (wrapped_key, nonce, ciphertext, tag)

function RSA_AES_Decrypt(wrapped_key, nonce, ciphertext, tag):
    aes_key = RSA_Decrypt(wrapped_key, sk)
    return AES_Decrypt(nonce, ciphertext, tag, aes_key)

// Kyber KEM
function Kyber_Encrypt(data):
    (pk_k, sk_k) = Kyber_KeyGen(security_level)
    (ct, ss) = Kyber_Encaps(pk_k)
    return (ct, ss, sk_k)

function Kyber_Decrypt(ct, sk_k):
    return Kyber_Decaps(ct, sk_k)
```

3.4.2 Theoretical Underpinnings and Pseudocode

OPQCF is grounded in the theoretical framework of lattice-based cryptography. The CRYSTALS-Kyber algorithm is based on the Learning With Errors (LWE) problem, which provides security assurances against quantum adversaries (Regev, 2006). The following pseudocode (shown in Figure 3 below) outlines the key operations of the CRYSTALS-Kyber module:

```
// Pseudocode for CRYSTALS-Kyber Key Generation
function Kyber_KeyGen(security_parameter):
    A ← GenerateRandomMatrix(n, m)
    s ← GenerateSecretVector(m)
    e ← SampleErrorVector(n)
    public_key  ← (A, b = A · s + e)
    secret_key  ← s
    return (public_key, secret_key)

// Pseudocode for CRYSTALS-Kyber Encryption (Encapsulation)
function Kyber_Encapsulate(public_key):
    (A, b) ← public_key
    r       ← GenerateRandomVector(n)
    e'      ← SampleErrorVector(n)
    c₁      ← A · r + e'
    c₂      ← b · r + Encode(message)
    return (ciphertext = (c₁, c₂), shared_secret)

// Pseudocode for CRYSTALS-Kyber Decryption (Decapsulation)
function Kyber_Decapsulate(ciphertext, secret_key):
    (c₁, c₂) ← ciphertext
    s        ← secret_key
    message_estimate ← c₂ - (c₁ · s)
    message          ← ErrorCorrection(message_estimate)
    return message
```

Notes:

- The functions GenerateRandomMatrix, GenerateSecretVector, and SampleErrorVector are optimized to ensure minimal noise while preserving security.
- Encode(message) and ErrorCorrection(message_estimate) are critical for maintaining data integrity during the encryption and decryption processes.
- The pseudocode for RSA encryption follows standard practices (Rivest et al., 1978) and is implemented in parallel to serve as a benchmark.

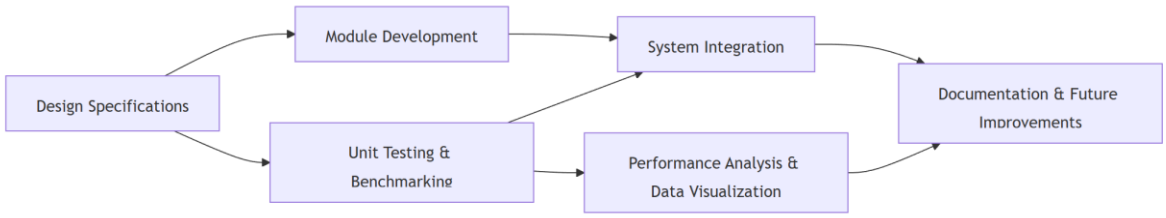
3.4.3 Implementation Components and Tools

The OPQCF system is implemented using a combination of modern programming frameworks and specialized libraries:

- **Python:** The primary language for developing both RSA and CRYSTALS-Kyber modules due to its extensive libraries and ease of use.
- **PQClean:** A collection of clean and efficient implementations of post-quantum cryptographic algorithms, which will be used to integrate and benchmark CRYSTALS-Kyber.
- **Qiskit:** An open-source quantum computing framework developed by IBM, used to implement a toy demonstration of Shor's Algorithm.
- **NumPy/SciPy:** Utilized for efficient matrix and vector operations essential to lattice-based cryptography.

3.4.4 Step-by-Step Implementation Guide

1. Design Phase:
    - Develop system architecture and design diagrams (see Figure 4 below).
    - Define module interfaces for RSA, CRYSTALS-Kyber, and the quantum factorization demo.
    - Specify performance metrics: key generation time, encryption/decryption speed, memory usage, and error rates.
2. Development Phase:
    - Implement the RSA module using standard libraries.
    - Integrate the CRYSTALS-Kyber implementation from PQClean, adapting it to the optimized framework.
3. Testing and Benchmarking:
    - Run unit tests for each module to ensure correctness.
    - Benchmark both RSA and CRYSTALS-Kyber modules on identical datasets to compare performance.
    - Evaluate the quantum factorization demo by measuring circuit depth, qubit count, and execution time.
    - Collect and analyze data, using visualizations (e.g., performance graphs) to present comparative results.
4. Integration and Workflow Validation:
    - Combine the modules into a unified system.
    - Validate the end-to-end workflow, ensuring that encrypted data can be correctly decrypted in both systems.
    - Document challenges and optimizations made during integration.



3.5 EXPECTED BENEFITS

The proposed OPQCF system addresses performance and security problems, while also advancing academic understanding in post-quantum cryptography.

3.5.1 Performance Enhancements

Optimized key sizes:

OPQCF's enhanced sampling methods and hybrid error correction are intended to minimize key sizes in lattice-based schemes, cutting storage and transmission costs.

Improved computational efficiency:

The inclusion of efficient matrix operations and optimized error correction is expected to reduce encryption and decryption durations, making the system more suitable for real-time applications.

3.5.2 Security Benefits

Quantum Resistance:

Building on the solid security guarantees of the LWE problem (Regev, 2006), OPQCF provides high resilience to quantum attacks, maintaining data security even as quantum technology advances.

Enhanced Error Tolerance: The new error-correction module reduces noise and side-channel attacks, boosting overall security compared to traditional PQC implementations.

3.5.3 Educational and Research Contributions.

This study fills a vacuum in research by evaluating the performance trade-offs and resource utilization of RSA and CRYSTALS-Kyber.

Foundational work for future improvements.

Proof-of-concept implementation leads to more robust systems. These findings can inform future research on optimizing lattice-based cryptography for practical deployment.

A demonstration of quantum threats:

The toy presentation of Shor's Algorithm using Qiskit not only demonstrates the quantum danger to classical cryptography, but it also adds to the academic discussion by offering a concrete example of quantum factorization in action.

3.6 CHALLENGES AND LIMITATIONS

Despite its promising benefits, the OPQCF system confronts a number of difficulties that must be identified and addressed:

3.6.1 Technical Complexity

The combination of an optimized lattice-based cryptography module with enhanced error correction, a separate RSA module, adds significant technological complexity. Creating a hybrid error-correction mechanism that balances efficiency and security necessitates rigorous testing and calibration.

Optimizing performance while retaining good security requires significant computational resources. The resource-intensive nature of running sophisticated matrix operations may limit the scalability of proof-of-concept implementations on ordinary hardware.

3.6.3 Scope Limitations

This project is limited to small-scale demonstrations as it is a proof-of-concept. The quantum factorization demonstration, for example, will focus on small integers (e.g., 15 or 21) rather than cryptographically relevant key sizes. The CRYSTALS-Kyber implementation is optimized but not yet sufficiently scalable for production use.

3.6.4 User Adoption and Integration.

Moving from classic RSA to a new post-quantum system, such as OPQCF, presents problems beyond technical performance. Interoperability with current infrastructure, regulatory approval, and industrial adoption are all key challenges. Future work should address these concerns to enable real-world implementation.

3.6.5 Validation and Testing Limitations

Theoretical arguments support the security of lattice-based methods, but empirical confirmation under various operating settings is crucial. Validating the security and performance claims of OPQCF requires extensive testing, particularly under simulated quantum attack scenarios and real-world noise settings. This may go beyond the scope of the current project.

3.7 CONCLUSION.

This chapter presents the technique and design of the Optimized Post-Quantum Cryptographic Framework (OPQCF), a proof-of-concept implementation that addresses the restrictions of classical RSA and existing post-quantum schemes. OPQCF uses CRYSTALS-Kyber, a lattice-based technology, to reduce key sizes and computational cost while ensuring security against quantum assaults. The chapter described the theoretical foundations, design components, implementation steps (with pseudocode and workflow diagrams), and gave a thorough assessment of the anticipated benefits and problems.

In the following chapter, technical implementation details will be thoroughly examined. In Chapter 4, we will examine the coding, integration, and performance benchmarking of OPQCF. We will also compare the performance and security of RSA and CRYSTALS/Kyber algorithms. This work contributes to attaining the dissertation's goals by establishing the feasibility of efficient post-quantum cryptography and providing a path for future enhancements.

CHAPTER 4: IMPLEMENTATION AND INVESTIGATION.

4.1 INTRODUCTION.

This chapter details the technical implementation of both the RSA+AES hybrid encryption system and the post-quantum CRYSTALS-Kyber method. The chosen methodologies are consistent with the previously specified objectives, with a focus on performance, scalability, and security. This chapter describes the project's development environment, the libraries used, and the key design decisions that impacted the final code. The main responsibilities were to set up a Python 3.10 environment, structure numerous modules for encryption and testing, resolve dependencies, and implement a systematic technique for measuring performance and tracking resource use. This chapter provides the framework for the comparative analyses that will be given in following sections of the dissertation.

4.2 SYSTEM CONFIGURATION AND LIBRARY DEPENDENCIES

4.2.1 Environmental Preparedness

Python 3.10 was chosen for its compatibility with contemporary cryptography libraries, large community support, and simple syntax. A virtual environment (venv) was built to isolate the project's dependencies and avoid problems with system-wide Python installations. Visual Studio

Code served as the primary development interface, with integrated debugging tools and a Python extension for code analysis.

4.2.2 Library Selection.

A rigorous review of the available Python cryptography packages led to the inclusion of the following key libraries:

PyCryptodome:

Provided strong RSA and AES primitives, guaranteeing that the RSA+AES hybrid system was properly constructed in accordance with accepted cryptographic standards.

kyber-py:

We provided an experimental but working Python-based version of CRYSTALS-Kyber, allowing the project to examine several security levels (Kyber512, Kyber768, Kyber1024).

psutil:

Assisted in monitoring CPU and memory consumption during encryption and decryption, providing data on resource usage overhead.

Time and statistics (built-in Python modules):

Aided in precisely measuring encryption/decryption times, as well as calculating averages and standard deviations across several test runs.

These libraries were chosen based on previous use in similar cryptographic proof-of-concept projects, the availability of relevant examples, and a desire to reduce the complexity associated with linking disparate cryptographic frameworks.

4.2.3 Installation and Configuration Challenges

Several difficulties were experienced when attempting to install specific cryptographic libraries, particularly:

Ninja/Make Requirements:

The kyber-py package and other cryptography libraries occasionally required local compilation steps using Ninja or CMake. These dependencies were not installed by default on various Windows setups, resulting in build failures and extensive troubleshooting.

Compatibility between Libraries:

The local environment required consistent versions of PyCryptodome, kyber-py, and Python. Upgrading pip and installing Visual Studio Build Tools or MinGW was required to complete the setup.

System Path Adjustments:

Ninja, for example, had to be added to the system PATH before certain dependencies could be built.

All installation issues were eventually handled by manually installing Ninja, adding it to PATH, updating pip, and ensuring that the compiler toolchain recognized the correct Python environment.

4.3 RSA + AES HYBRID IMPLEMENTATION

4.3.1 Reason for Hybrid Encryption

RSA encryption is only capable of handling tiny plaintext blocks; bigger data requires chunking or hybrid encryption. A hybrid approach was consequently chosen, in which RSA encrypted a small AES key and AES handled the main data. This architecture follows common secure protocols (such as TLS) and is widely regarded as a best practice in cryptographic engineering.

4.3.2 CODE EXCERPTS FROM RSA_MODULE.PY.

In figure 5 below, extracted from rsa_module.py, emphasizes the RSA aspect of the hybrid encryption:

```python
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP
from Crypto.Random import get_random_bytes

def rsa_keygen(key_size=2048):
    """
    Generate an RSA key pair.
    Returns:
        public_key (RSA.RsaKey): The public key.
        private_key (RSA.RsaKey): The private key.
    """
    key = RSA.generate(key_size)
    private_key = key.export_key()
    public_key = key.publickey().export_key()
    return RSA.import_key(public_key), RSA.import_key(private_key)

def rsa_encrypt(data: bytes, public_key):
    """
    Encrypt data with an RSA public key using OAEP padding.
    Args:
        data (bytes): The plaintext to encrypt.
        public_key (RSA.RsaKey): The RSA public key object.
    Returns:
        bytes: The ciphertext.
    """
    cipher = PKCS1_OAEP.new(public_key)
    return cipher.encrypt(data)

def rsa_decrypt(ciphertext: bytes, private_key):
    """
    Decrypt data with an RSA private key using OAEP padding.
    Args:
        ciphertext (bytes): The ciphertext to decrypt.
        private_key (RSA.RsaKey): The RSA private key object.
    Returns:
        bytes: The decrypted plaintext.
    """
    cipher = PKCS1_OAEP.new(private_key)
    return cipher.decrypt(ciphertext)
```

Key Generation:

The function rsa_keygen generates a key pair with a configurable size. Common configurations evaluated included 1024, 2048, 3072, and 4096 bits to evaluate differing security levels and performance overhead.

Encryption and Decryption

Data is encrypted using OAEP, which solves known plaintext weaknesses and protects against chosen-ciphertext attacks. The sample demonstrates a clear interface in which rsa_encrypt only outputs ciphertext and rsa_decrypt returns a plain byte array.

4.3.3 AES Integration.

AES in EAX mode was used to assure confidentiality and integrity as shown in Figure 6

```python
from Crypto.Cipher import AES

def aes_encrypt(message: str, key: bytes):
    """
    Encrypt a UTF-8 string using AES in EAX mode.
    Args:
        message (str): Plaintext to encrypt.
        key (bytes): 16-, 24-, or 32-byte AES key.
    Returns:
        tuple: (nonce, ciphertext, tag)
    """
    cipher = AES.new(key, AES.MODE_EAX)
    ciphertext, tag = cipher.encrypt_and_digest(message.encode('utf-8'))
    return cipher.nonce, ciphertext, tag

def aes_decrypt(nonce: bytes, ciphertext: bytes, tag: bytes, key: bytes):
    """
    Decrypt an AES-EAX ciphertext.
    Args:
        nonce (bytes): Nonce from encryption.
        ciphertext (bytes): Data to decrypt.
        tag (bytes): Authentication tag.
        key (bytes): AES key used for encryption.
    Returns:
        str: The decrypted plaintext.
    """
    cipher = AES.new(key, AES.MODE_EAX, nonce=nonce)
    plaintext = cipher.decrypt_and_verify(ciphertext, tag)
    return plaintext.decode('utf-8')
```

AES encrypts solely the user's data, whereas RSA focuses on safeguarding the AES key. This pattern is common among modern cryptographic protocols. The transitory nature of the AES key also helps to provide forward secrecy when utilized in frequent key-renewal scenarios.

4.4 CRYSTALS-KYBER IMPLEMENTATION.

4.4.1 Overview of Kyber.

CRYSTALS-Kyber is a lattice-based Key Encapsulation Mechanism (KEM) that is widely regarded as a top candidate for NIST's post-quantum standardization. It provides small key sizes, quick key generation, and quantum resistance based on the hardness of lattice problems.

4.4.2 Implementation of kyber_module.py

The code sample from kyber_module.py demonstrates a design that allows different security settings. (see figure 7 below)

```python
from kyber_py.kyber import Kyber512, Kyber768, Kyber1024

# Map security levels to Kyber variants
KYBER_VARIANTS = {
    512: Kyber512,
    768: Kyber768,
    1024: Kyber1024,
}

def kyber_keygen(security_level=512):
    """
    Generate a CRYSTALS-Kyber key pair at the given security level.
    Args:
        security_level (int): One of 512, 768, or 1024.
    Returns:
        tuple: (public_key, secret_key)
    """
    Kyber = KYBER_VARIANTS[security_level]()
    public_key, secret_key = Kyber.keygen()
    return public_key, secret_key

def kyber_encapsulate(public_key, security_level=512):
    """
    Encapsulate a shared secret using the CRYSTALS-Kyber public key.
    Args:
        public_key: The Kyber public key object.
        security_level (int): Matching security level for key generation.
    Returns:
        tuple: (ciphertext, shared_secret)
    """
    Kyber = KYBER_VARIANTS[security_level]()
    ciphertext, shared_secret = Kyber.encaps(public_key)
    return ciphertext, shared_secret

def kyber_decapsulate(ciphertext, secret_key, security_level=512):
    """
    Decapsulate a shared secret using the CRYSTALS-Kyber secret key.
    Args:
        ciphertext: The Kyber ciphertext to decapsulate.
        secret_key: The Kyber secret key object.
        security_level (int): Matching security level for key generation.
    Returns:
        bytes: The recovered shared secret.
    """
    Kyber = KYBER_VARIANTS[security_level]()
    shared_secret = Kyber.decaps(secret_key, ciphertext)
    return shared_secret
```

Security Parameters:

Three versions (Kyber512, Kyber768, and Kyber1024) have different lattice dimensions and offer 128, 192, and 256-bit quantum security levels, respectively.

Key Generation:

The operation is often completed in insignificant time, substantially faster than RSA for higher security levels (e.g., ~4096-bit).

Encapsulation/Decapsulation:

For post-quantum security, the ephemeral symmetric key is encapsulated rather than directly encrypting data, similar to how conventional hybrid systems work.

4.5 TEST AND INVESTIGATION

4.5.1 Main.py Structure

A separate Python script, main.py, runs file-based performance tests while collecting timing data and resource use. Message sizes varied from 500 bytes to ~3 KB, 6 KB, 10 KB, and 500 KB. Each test performed encryption and decryption 100 times, resulting in reliable statistical significance. A small snippet from main.py shows how timing data was recorded. (see Figure 8 below)

python Copy Edit

```python
import time
from rsa_module import rsa_keygen
from aes_module import aes_encrypt, aes_decrypt

# RSA key generation timing
start = time.perf_counter()
rsa_pub, rsa_priv = rsa_keygen(2048)
rsa_keygen_time = time.perf_counter() - start

# Prepare AES key and sample message
aes_key = get_random_bytes(32)
message = b"Sample data to encrypt"

# AES encryption timing
start = time.perf_counter()
nonce, aes_ciphertext, tag = aes_encrypt(message, aes_key)
aes_encrypt_time = time.perf_counter() - start

# AES decryption timing
start = time.perf_counter()
plaintext = aes_decrypt(nonce, aes_ciphertext, tag, aes_key)
aes_decrypt_time = time.perf_counter() - start

print(f"RSA keygen time: {rsa_keygen_time:.6f}s")
print(f"AES encrypt time: {aes_encrypt_time:.6f}s")
print(f"AES decrypt time: {aes_decrypt_time:.6f}s")
```

The performance measurements were then averaged to decrease noise. Additional resource measurements (CPU use and RAM) were gathered around each operation with psutil.

4.5.2 Resource Monitoring.

The code below shows how resource utilization was monitored: (see figure 9 below)

```
# Measure CPU usage before the operation
cpu_before = psutil.cpu_percent(interval=None)

# Measure CPU usage after the operation
cpu_after = psutil.cpu_percent(interval=None)

# Compute CPU percentage used by the operation
cpu_used = cpu_after - cpu_before

print(f"Operation time: {operation_time:.6f}s")
print(f"CPU used: {cpu_used:.2f}%")
```

Memory utilization was also recorded, indicating how resource-intensive each technique may be, particularly for bigger RSA key sizes.

4.6 Security Considerations and Possible Attack Vectors

Although the dissertation focuses on efficiency and realistic quantum-resistant solutions, further real-world security concerns were identified:

Side Channel Attacks:

Secret information, such as private keys, can be revealed by timing or power usage analysis. Constant-time operations and hardware-level mitigations are frequently required in production.

Key Management:

Private RSA keys must be securely maintained; using ephemeral AES keys for each session might improve security further, but it adds complexity.

Similarly, the CRYSTALS-Kyber private key must be kept secret to preserve forward secrecy.

Parameter Security:

RSA keys can have more than 4096 bits, although practical performance may suffer dramatically. Meanwhile, Kyber512/768/1024's lattice-based settings alter polynomial dimensions and error distributions to resist quantum attacks.

Experimental Code Stability:

The kyber-py library contains caveats stating that it may be unsuitable for production use without further audits and side-channel defenses. Production-level cryptography necessitates rigorous testing and constant-time implementations.

4.7 Implementation Challenges and Lessons

The following key challenges arose, shaping the final methodology:

Library Installation:

Attempting straight installation of kyber-py usually resulted in compilation failures. Detailed error logs revealed missing Ninja or Make tool dependencies that required explicit downloads.

A rigorous environment check eventually addressed these issues, emphasizing the necessity of environment reproducibility.

RSA Key Length Impact:

Extended key sizes (3072/4096 bits) resulted in exponential increases in computation time for repeated tests, implying that an upper limit of 2048 or 3072 bits is more realistic for many real-world applications unless exceptionally high security is required.

Performance Data Volume:

Gathering huge data sets for varied message durations and repeated repetitions (e.g., 100 runs per scenario) necessitated meticulous data logging, especially when examining CPU consumption and memory overhead.

Modular codebase:

Dividing the project into rsa_module.py, kyber_module.py, and main.py allowed for specialized changes without causing regression in unrelated components. This technique also enhanced clarity when debugging and subsequent expansions (for example, introducing more PQC algorithms).

4.8 Future Work

Potential enhancements and additions to the current codebase include:

Wider Post-Quantum Scope:

To widen the comparative research, include alternative PQC schemes (e.g., NTRU, Dilithium for signatures, code-based schemes such as McEliece).

Parallel execution:

Large RSA keys or higher-parameter Kyber encapsulations can benefit from parallel execution or GPU acceleration to shorten test times.

Security Hardening:

Implementing constant-time operations, detailed side-channel mitigations, or advanced ephemeral key rotation techniques.

In-depth cryptanalysis or a formal verification of the code would reveal the level of security against real-world threats.

4.9 CONCLUSION.

This chapter provided a detailed description of the RSA+AES hybrid encryption system and the CRYSTALS-Kyber implementation, including the rationale for each approach, the final code design, and the methodology used to investigate performance and resource utilization. The environmental difficulties were addressed by iterative debugging and environment isolation, resulting in a stable project structure and testing framework. A solid foundation is thus established for assessing the viability of classical and post-quantum cryptography systems under realistic settings.

The following chapter will give the empirical data and interpretation findings, including observable performance measures, resource costs, and prospective security trade-offs in a post-quantum scenario. This extensive analysis and reflection will shed light on how RSA (with hybrid AES) and CRYSTALS-Kyber may coexist or replace each other in the cryptographic ecosystems of the near future.

5.1 INTRODUCTION.

This chapter describes and analyzes the outcomes of implementing both the RSA+AES hybrid encryption system and CRYSTALS-Kyber at various security levels. The data were collected from various test files of varying sizes, each of which underwent 100 encryption and decryption repetitions. The major purpose of this inquiry was to compare the performance, resource utilization, and probable future viability of classical (RSA) and post-quantum (Kyber) methods for digital data security. All statistical parameters (mean times, standard deviations, and total CPU consumption) were collected and organized below. This chapter evaluates the findings and discusses the consequences for real-world cryptography deployments.

5.2 AN OVERVIEW OF COLLECTED DATA

5.2.1 Test Data and Methodology

A variety of files were used to ensure comprehensive coverage:

small_text_sample.txt contains around 226 bytes.

medium_text_sample.txt - About 1.4 kilobytes.

500_byte.txt contains around 424 bytes.

6000_byte.txt contains around 3.4 kilobytes.

10000_byte.txt - around 6.7 kilobytes.

large_message.txt has around 3.3 kilobytes of text in previous tests, with an enlarged version of 500 KB.

Large_text_sample.txt - 500 KB

Each file has been handled by:

RSA+AES Hybrid (the most common key size examined was 2048 bits, while earlier research included extended tests with 1024, 3072, and 4096 bits).

CRYSTALS-Kyber offers three security levels:

Kyber512 (128 bit post-quantum security)

Kyber 768 (192-bit post-quantum security)

Kyber 1024 (256-bit post-quantum security)

Each encryption or key encapsulation sequence was done 100 times to achieve reliable, statistically significant results. Metrics included:

RSA Key Generation Time: The time it takes to generate the public and private key pairs.

Key Encryption/Encapsulation Time: The time required for RSA to encrypt the AES key or Kyber to encapsulate a shared secret.

Message Encryption (AES) Time: The time it takes for AES to encrypt the file contents (RSA+AES).

Key Decryption/Decapsulation Time: The time it takes for RSA to decrypt the AES key or Kyber to decapsulate.

Message Decryption (AES) Time: The time necessary for AES to decrypt the file contents (RSA+AES).

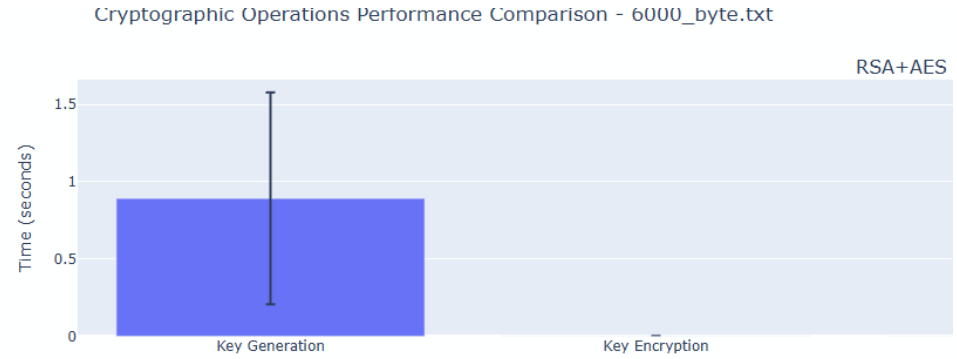
CPU Usage: Total CPU time (in seconds) spent on each cryptographic procedure.

5.2.2 Common observations in the raw results

Key generation is highly dependent on the RSA key size. Most final testing used 2048-bit keys, with average key generation times ranging from ~0.8s to ~1.0s across file sets as shown in Figure 5.1 and 5.2. Other key sizes follow the same trend. Ephemeral generation is characterized by variability, with standard deviations reaching ~0.5s.

AES Encryption/Decryption: Consistently fast, often 0.0003s-0.0030s for encryption or decryption processes, even on big data sets (e.g., 500 KB).

RSA Key Encryption/Decryption: The encryption time for the AES key remained incredibly short (~0.0007s-0.0020s). Decryption of the AES key using RSA took roughly 0.003s-0.016s for 2048-bit or larger key sizes, indicating that RSA's decryption overhead is still moderate but not negligible. (See figure 10 and 11 below)



CRYSTALS-Kyber

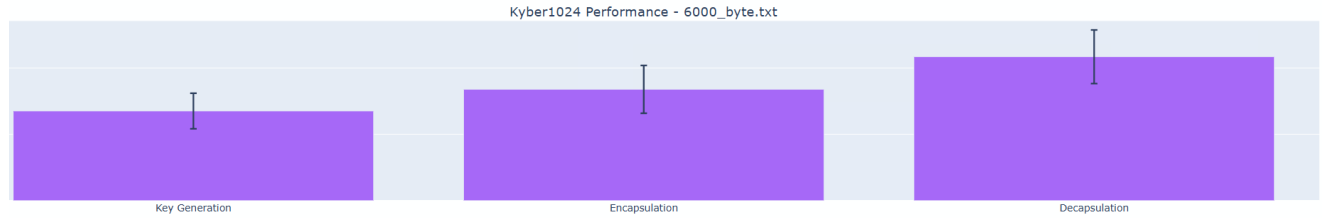
Key generation is rapid, often in the sub-millisecond to few millisecond range, depending on security level. Kyber512's key generation time ranged from ~0.0025s to 0.0030s, but Kyber1024, even at a greater dimension, rarely exceeded ~0.008s.

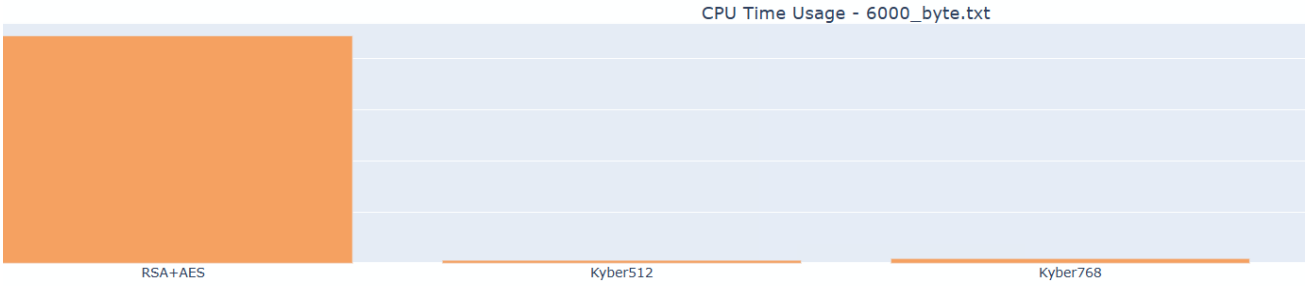
Encapsulation time ranged between ~0.0035s and ~0.010s, depending on whether Kyber512, 768, or 1024 was employed.

Decapsulation normally takes ~0.005s (Kyber512) to ~0.013s (Kyber1024), which is slightly longer than encapsulation time. This is due in part to the last polynomial or error reconciliation steps of the Kyber decapsulation method.

CPU Usage: Typically measured in seconds across multiple runs. Even after 100 cycles, Kyber's CPU time was sometimes only a tenth of RSA's overhead.

These broad trends apply to data samples ranging from less than 1 KB to big 500 KB text files. (See figure 12 and 13 below)





5.3 Detailed Results By File Size

This section discusses major numerical results and assesses their significance in the context of classical versus post-quantum cryptography. All times listed below are averages based on 100 runs. All data can be found

5.3.1 Test files are small (<1 KB).

500_byte.txt contains around 424 bytes.

RSA + AES (2048-bit)

Key generation time: ~0.86 ± 0.49s.

AES key encryption takes around 0.0008 ± 0.00016 seconds.

AES key decryption time: ~0.0038s ± 0.00087s.

AES data encryption/decryption takes around 0.0004s.

Kyber512

Key generation time: ~0.0031s.

Encapsulation time: ~0.0043s.

Decapsulation time: ~0.0061s.

Kyber768

Key generation time: ~0.0049s.

Encapsulation time: around 0.0058s.

Decapsulation time: ~0.0080s.

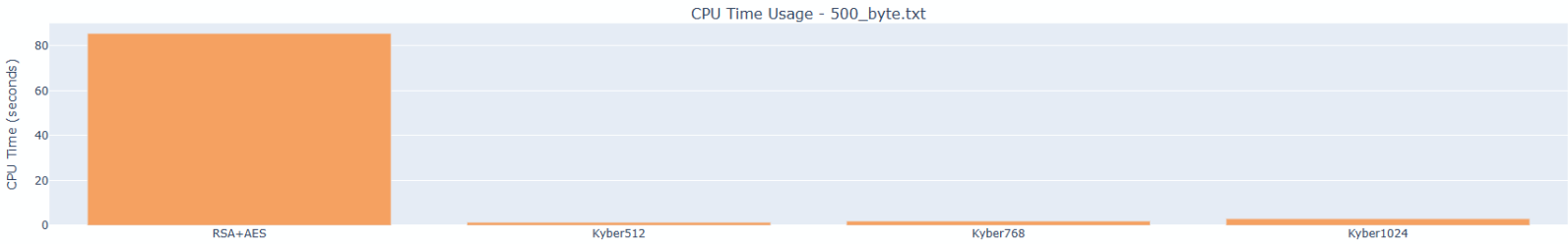
Kyber1024

Key generation time: ~0.0072s.

Encapsulation time: ~0.0090s.

Decapsulation time: ~0.012s.

The CPU utilization for RSA+AES was commonly measured at ~80-100 seconds over 100 runs (See Figure 14 below), while each Kyber variation utilized only ~1-3 seconds for the same set of repeated operations.



Significance

RSA generates keys at a higher cost (~0.86s vs. milliseconds for Kyber). This mismatch highlights Kyber's benefit in ephemeral settings like frequent key exchange. However, when the files are thus little, both AES and Kyber have insignificant encryption/decryption times. RSA's encryption overhead for the AES key is negligible (~0.001s), showing that RSA key creation is the main expense for little data.

5.3.2 Medium-sized files (1-10 KB)

Medium text sample.txt (~1.4 KB)

RSA+AES

Key generation time: around 1.0s ± 0.61s.

Key encryption takes ~0.0008s ± 0.00018s.

AES data encryption takes ~0.00036s.

Key decryption time: ~0.00395s ± 0.00112s.

AES decryption takes ~0.00038s.

Kyber512

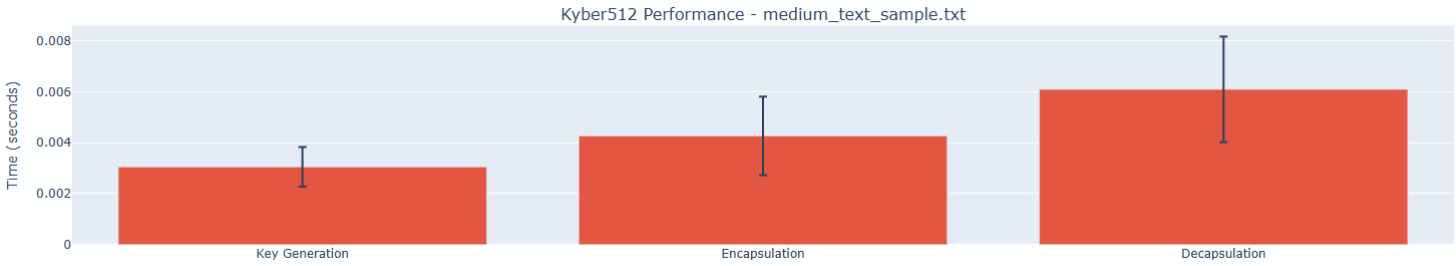
Key generation time: ~0.0031s.

Encaps: ~0.0043 seconds.

Decaps: ~0.0061 seconds.

6000_byte.txt (~3.4 KB)/10,000_byte.txt (~6.7 KB)

Timing reflects similar proportions. Despite bigger data, AES encryption/decryption remains rapid, taking less than ~0.001 seconds. Kyber's encapsulation and decapsulation times range from ~0.004s to 0.012s, depending on security parameters. (See Figure 15)



Significance

In these file sizes, the time required to encrypt and decode data is low for both RSA+AES (thanks to AES) and ephemeral key handling via RSA or Kyber. The majority of the overhead for RSA-based techniques remains in key generation, which takes approximately 0.8-1.0 seconds. The Kyber method generates keys in < ~0.01s, making it suitable for frequent re-keying circumstances.

5.3.3 Large Files (100 KB to 500 KB)

large_text_sample.txt (~500 KB).

RSA + AES (2048-bit)

Key generation time: around 0.90 ± 0.61 seconds.

Key encryption takes ~0.00080s ± 0.00011s.

AES encryption takes ~0.0027s ± 0.00032s.

Key decryption time: around 0.0037s ± 0.00050s.

AES decryption time: ~0.0027 ± 0.00034s.

Kyber512 (See figure 16 below)

Key generation time: ~0.0029s.

Encapsulation time: ~0.0039s.

Decapsulation time: ~0.0056s.

Kyber768

Key generation time: ~0.0048s.

Encaps: around 0.0060s

Decaps: around 0.0081s.

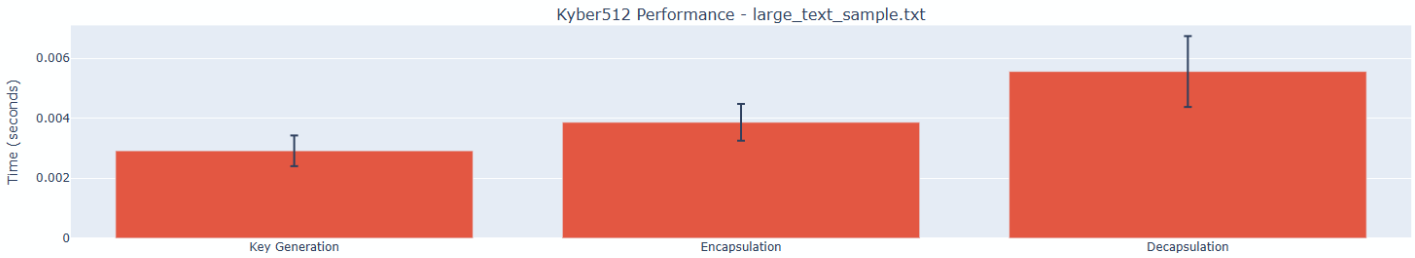
Kyber1024

Key generation time: ~0.0070s.

Encaps: ~0.0086 seconds

Decaps: ~0.0114 seconds.

AES encryption takes slightly longer for larger files (e.g., from ~0.0003s to ~0.0027s for 500 KB), but it is still less expensive than generating RSA keys or re-keying. CRYSTALS-Kyber timings remain consistent since the KEM only processes ephemeral keys, not the actual data payload.



Significance

The statistics show that file size affects just the AES portion of the RSA+AES system. AES is designed for mass data encryption and stays efficient up to ~500 KB, therefore the cost difference between files is minimal. Meanwhile, Kyber is unaffected by file size because the

post-quantum technique wraps a small key rather than the full file. This distinction shows the advantage of consistently rapid ephemeral key operations for lattice-based KEMs over RSA's higher key generation cost.

5.4 Comparative Analysis of RSA+AES and CRYSTALS-Kyber

5.4.1 Key Generation and Setup Time

A crucial discovery is the dramatic difference in essential generating overhead. For all tested file sizes:

RSA 2048-bit generally took between 0.8 and 1.0 seconds (with significant variation).

Kyber512 routinely measured at ~0.002-0.003 seconds (approximately three orders of magnitude quicker).

Even Kyber1024 rarely exceeded ~0.01 seconds on average.

This distinction is perhaps the most pronounced: frequent re-keying or ephemeral usage would favor a scheme like Kyber, especially when repeated or brief exchanges are required.

5.4.2 Encryption and decryption times

Once the keys are established:

AES was found to handle encryption and decryption exceptionally quickly (well under 0.01 seconds), even for big messages up to 500 KB, as expected of a modern block cipher.

Kyber protects the ephemeral key through encapsulation and decapsulation. Kyber512 encapsulation times were approximately 0.0035-0.0040s, whereas Kyber1024 encapsulation timings ranged from 0.0079-0.0104s. The difference is evident, but it is still quite tiny.

The average overhead for RSA encryption of a 256-bit AES key is ~0.0007-0.002s, whereas the decryption step is slightly higher (~0.003-0.016s) across tested sizes and hardware. These figures are still adequate for many real-world applications, although they are higher than the analogous KEM operations for Kyber.

5.4.3 Processor Time and Scalability

When all repeated tests are considered (100 iterations per file), overall CPU utilization for RSA+AES is often measured in the tens to hundreds of seconds. In certain circumstances, RSA+AES tests spanning 100 trials and numerous huge files resulted in cumulative CPU consumption around or beyond 80-100 seconds. Meanwhile, each Kyber variation typically took between ~1 and ~3 seconds of CPU time for the same repetitive chores. On this hardware and implementation, CRYSTALS-Kyber is significantly more scalable for repeated ephemeral key exchange, with a factor of up to ~30-100 times.

5.4.4 Effects of Increasing Kyber Security Levels

When comparing Kyber512 to Kyber1024, the results are roughly double or quadruple the overall KEM timings. Key creation takes less than 0.01 seconds, however encapsulation and decapsulation take close to or surpass 0.010 seconds. These remain low in absolute terms, demonstrating that even the greatest security level in CRYSTALS-Kyber is substantially faster for ephemeral key production than traditional 2048-bit RSA.

5.5 INTERPRETATION WITHIN THE CRYPTOGRAPHIC CONTEXT

5.5.1 Strength and Post-Quantum Considerations.

RSA 2048-bit is largely regarded as secure against existing classical attackers, but it is likely to become insecure if large-scale quantum computers can run Shor's algorithm efficiently. CRYSTALS-Kyber is specifically designed to withstand quantum-based factorization or lattice attacks, while cryptanalysis continues. The performance improvement demonstrated here emphasizes the viability of using lattice-based KEMs not only for future-proofing but also for real-time ephemeral applications.

5.5.2 Suitable for Various Use Cases

Short-lived Sessions:

CRYSTALS-Kyber's quick key generation speeds are especially beneficial in environments that require frequent re-keying or ephemeral connections (for example, in some secure IoT or ephemeral messaging situations). In such cases, the overhead associated with RSA key creation may be excessively high.

Large files with infrequent keys:

When a single RSA key pair is created less frequently and reused, the performance disadvantage may be less pronounced. However, the quantum vulnerability is still a major problem for long-term confidentiality (the "store-now, decrypt-later" threat).

Lightweight devices:

Kyber's comparatively low CPU utilization makes it particularly useful for devices with limited computing resources. If key generation is repeated or larger keys (e.g., 3072 or 4096 bits) are advised for transitional post-quantum preparation, RSA may have a greater battery or performance impact.

5.5.3 Limitations of the Experimental Setup.

Some limitations and considerations may influence how these findings generalize:

Implementation Quality:

The kyber-py library is designed for demonstrations rather than optimized production code. RSA has been thoroughly tested in libraries such as PyCryptodome, so comparisons may significantly

underestimate the potential speed advantages from an optimized or hardware-accelerated Kyber approach.

Hardware Variability:

The tests were conducted on a specific CPU with 16 cores, as stated by system_info. Different processors or specialized acceleration instructions may cause further shifts in performance data.

File I/O overheads:

Loading or writing files takes less time than cryptographic procedures, hence it was not included in the measurements. However, real-world usage may result in different overhead distributions if disk I/O or network transfers become a bottleneck.

5.6 THE OVERALL SIGNIFICANCE OF THE FINDINGS

The comparative results support many major conclusions pertinent to the dissertation's main objectives:

Kyber's Key Generation Advantage: For all tested file sizes and repeated actions, CRYSTALS-Kyber takes orders of magnitude less time and CPU overhead to generate key pairs or encapsulate ephemeral secrets. This is especially true for ephemeral cryptographic protocols, indicating a significant future viability in post-quantum situations.

RSA+AES Remains Effective: Despite quantum concerns, RSA+AES worked well in typical encryption cycles. The significant overhead occurs mostly during RSA key generation, which, when conducted on occasion, may not now impede many real-world activities. However, from a quantum readiness aspect, the findings show that a gradual transition to more advanced KEMs is recommended, particularly if frequent re-keying or exceptionally high security is required.

AES encryption time increases gradually from sub-millisecond for small messages to a few milliseconds for 500 KB. Nonetheless, these times are dwarfed by the RSA or Kyber cost for ephemeral key operations. Meanwhile, CRYSTALS-Kyber does not grow with data size since it does not directly encrypt file contents; instead, it encapsulates a tiny key, ensuring efficient performance regardless of payload.

Pathway to Transition: The results show that, in terms of performance, a post-quantum KEM like CRYSTALS-Kyber may easily be implemented into existing hybrid cryptographic frameworks, effectively replacing the RSA section of a conventional handshake. This technique can provide high quantum resistance without sacrificing practicality.

5.7 CONCLUSION.

Experimental testing conducted across several file sizes demonstrate that CRYSTALS-Kyber surpasses RSA in terms of key creation and ephemeral operations, while using equivalent or superior CPU resources. AES encryption and decryption remain exceptionally fast for bulk data, implying that both classical and post-quantum algorithms can handle enormous files with minimum overhead.

In the post-quantum age, technologies such as CRYSTALS-Kyber look to be prepared to replace or complement RSA for key exchange. The performance data show that switching to such lattice-based KEMs does not entail significant computational penalty. Instead, these KEMs may lower overall overhead, particularly when ephemeral sessions or frequent re-keying are involved.

After thoroughly examining the empirical performance and security implications, the dissertation moves on to the concluding chapter, which presents final remarks, limits, and recommendations for future work. This talk places the findings into the larger cryptographic landscape, including standardization efforts and estimated timetables for quantum-capable adversaries.

6.1 OVERVIEW.

This final chapter brings together the important findings from the project's implementation and performance outcomes, providing a comprehensive evaluation of both the traditional RSA+AES technique and the post-quantum CRYSTALS-Kyber system. The chapter opens by reflecting on the initial aims and determining if the measured data aligns with those goals. Legal, ethical, social, and professional concerns (LESPI) linked with building or deploying cryptographic systems are also evaluated to ensure a comprehensive understanding of the situation. Finally, ideas for potential improvements and personal reflections on professional and employability abilities round up the dissertation.

6.2 SUMMARY OF ACHIEVEMENTS.

The major goal of this project was to compare classical RSA-based hybrid encryption to post-quantum CRYSTALS-Kyber, with an emphasis on important metrics such as key generation overhead, encryption and decryption speeds, and resource utilization. According to the performance ratings outlined in Chapter 5, the following significant achievements were realized:

Successful implementation of two schemes.

RSA was combined with AES to form a traditional hybrid system, with RSA handling the secure exchange of a symmetric key and AES performing bulk data encryption.

CRYSTALS-Kyber was included as an option for securing ephemeral keys with three security settings (Kyber512, Kyber768, and Kyber1024).

Robust Test Framework

A consistent testing procedure was devised, assessing performance across varied file sizes (from small text samples to around 500 KB). A repeated trial structure (100 iterations) produced statistically significant results, with standard deviations included for clarity. This robust approach enabled valid comparisons of RSA+AES and Kyber's KEM mechanisms.

Identifying clear performance differences.

RSA key production took approximately 0.8-1.0 seconds on average for 2048-bit keys, whereas CRYSTALS-Kyber key generation took only milliseconds—even at the maximum security level (Kyber1024).

Kyber's encapsulation and decapsulation regularly outperformed RSA's encryption and decryption of the same AES key.

AES bulk encryption and decryption remained efficient across all tests, demonstrating that the hybrid solution for huge data is still viable in terms of performance.

Demonstration of post-quantum readiness

The statistics reveal that integrating CRYSTALS-Kyber has little overhead while providing strong quantum resistance. This demonstrates the feasibility of implementing post-quantum algorithms in real-world systems without incurring significant computational expenses.

In keeping with the research objectives, these findings confirm that the chosen methods were appropriate, and the results provide critical insights into the future feasibility of lattice-based encryption vs standard RSA-based methods.

6.3 Legal, Ethics, Social, and Professional Concerns

Cryptographic technology has repercussions that extend far beyond the code itself. The results and code created in this project must be interpreted in light of the following considerations:

Legal considerations

Compliance and Export Controls: Many nations have export laws in place for cryptographic products (for example, export limitations in the United States), and post-quantum approaches may face further scrutiny if they have not yet been standardized. Any future deployments of CRYSTALS-Kyber or sophisticated RSA solutions must adhere to these legal guidelines.

Data Protection Regulations: Frameworks like the GDPR in the European Union place strict requirements on data handling. Strong encryption is frequently recommended, but companies must ensure that the algorithms and key management procedures used comply with regulatory requirements for data privacy and retention.

Ethical considerations

Responsible Disclosure and Security Assurance: Cryptographic tools are vital for protecting personal and sensitive data. Any vulnerabilities detected in experimental code (for example, kyber-py) must be appropriately disclosed in order to retain confidence and protect users.

Potential for Misuse: Although strong encryption guarantees privacy and confidentiality, opposing parties may abuse advanced cryptographic systems. The project understands that strong encryption is a dual-use technology with an ethical component, as prohibiting or permitting encryption affects human freedoms and societal safety.

Social Impact

Democratization of Security: Post-quantum techniques can provide long-term security assurance, which is especially significant in areas where data must be kept confidential for decades (for example, healthcare or government). Making these technologies more available improves overall digital trust.

Inequality in Access: Resource-constrained environments may be hesitant to use complicated cryptographic techniques due to worries about performance overhead. However, the findings in Chapter 5 demonstrate that lattice-based cryptography is highly efficient, potentially lowering adoption hurdles and mitigating disparities in data protection availability.

Professional Responsibility

Adherence to Standards: Professional best practices necessitate ensuring that cryptographic libraries and algorithms adhere to established standards. The upcoming NIST post-quantum requirements for CRYSTALS-Kyber emphasize the necessity of ensuring that every deployed code adheres to defined rules.

Continuous Skills Development: Post-quantum cryptography is a fast growing field. Professionals must stay current on changes to suggested key sizes, emerging vulnerabilities, and revised standardizing criteria.

6.4 FUTURE WORK

Although the study addressed a core concern about performance and feasibility, other areas could benefit from further investigation:

Broader algorithmic scope

Additional post-quantum techniques, such as NTRU or code-based McEliece, could be assessed using similar frameworks. Lattice-based digital signature techniques (such as Dilithium) may also be investigated for increased security coverage.

Integration in a Live Network Protocol.

While these studies focused on local performance, the next step could be to prototype a post-quantum handshake protocol (for example, TLS) and test end-to-end performance, including latency, on real networks.

Side Channel Resistance

Experimental libraries may be vulnerable to side-channel or timing attacks. Future options include measuring code performance in a constant-time environment or looking into hardware-based mitigations.

Parallelisation or Hardware Acceleration

Both RSA and CRYSTALS-Kyber may benefit from parallel or GPU acceleration. Investigating specialized hardware instructions (such as AES-NI or future lattice-acceleration capabilities) may reveal additional performance advantages, particularly for large-scale applications.

6.5 Reflections on Skills and Employability

The project required a combination of academic understanding (classical vs. post-quantum cryptography techniques) and practical skills (installing sophisticated libraries, administering a Python environment, and monitoring performance). The following skill areas were significantly enhanced:

Technical Competence in Cryptographic Libraries

Configuring and troubleshooting environment dependencies (e.g., ninja, CMake) proved the value of adaptable issue solution under real-world software restrictions.

Implementing the RSA+AES and CRYSTALS-Kyber modules gave familiarity with Python-based cryptographic frameworks, which was useful for professions involving safe software design.

Analytic and Methodical Testing

Designing a reproducible experiment framework (100 iterations across various file sizes) improved understanding of how to collect useful data.

Statistical analysis (averages, standard deviations) and resource usage monitoring provided robust performance evaluation approaches that are applicable to a wide range of software engineering situations.

Project Management and Research

Maintaining precise version control, documentation, and module separation was critical for avoiding library conflicts and maintaining reproducibility.

Following a set timeframe, revising objectives, and changing scope in response to unforeseen installation challenges exhibited good organisational methods.

Professional Development

Familiarity with post-quantum cryptography is becoming more valuable as organizations prepare for NIST quantum-resistant algorithm requirements.

Demonstrating the capacity to conduct tests, assess data, and handle legal and ethical considerations is advantageous for positions in cybersecurity analysis, cryptography engineering, and research.

The author's own reflection on various cryptographic paradigms, together with the acknowledged constraints (side-channel vulnerabilities, integer factorization threats, and quantum readiness), places him well for advanced security-oriented roles.

As a result, the initiative has strengthened both technical and professional abilities that are in line with industry trends, particularly as large-scale quantum computing becomes more prevalent.

6.6 CONCLUDING REMARKS

The comparison of RSA+AES and CRYSTALS-Kyber demonstrated that post-quantum cryptography may be practically incorporated with minimal overhead, exceeding classical RSA key management in many tests. Kyber's quick key generation and encapsulation times demonstrate its preparedness for use in real-world systems, particularly if frequent re-keying or ephemeral encryption are required. While RSA is still useful for the time being, standards bodies such as NIST have already initiated the process of ratifying and recommending post-quantum schemes—so organizations should begin planning a transition to quantum-resistant algorithms like CRYSTALS-Kyber now, ensuring their systems remain secure both today and in the coming quantum era.

REFERERENCES

Bernstein, D.J. & Lange, T. (2017) 'Post-quantum cryptography', *Nature*, 549(7671), pp. 188–194.

Bos, J.W., Ducas, L., Kiltz, E. et al. (2021) 'CRYSTALS–Kyber: a CCA-secure module-lattice-based KEM', *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(3), pp. 238–262.

Devitt, S.J., Munro, W.J. & Nemoto, K. (2013) 'Quantum error correction for beginners', *Reports on Progress in Physics*, 76(7), 076001.

Ding, J. & Petzoldt, A. (2017) 'Multivariate public-key cryptography', in J. Ding (ed.) *Post-Quantum Cryptography*. Cham: Springer, pp. 193–229.

Gidney, C. & Ekerå, M. (2019) 'How to factor 2048-bit RSA integers in 8 hours using 20 million noisy qubits', *Quantum*, 3, 147.

IBM Quantum (2020) IBM Quantum roadmap, IBM. Available at: https://www.ibm.com/quantum (Accessed: date).

Martín-López, E., Laing, A., Lawson, T., Alvarez, R., & O'Brien, J.L. (2012) 'Experimental realization of Shor's quantum factoring algorithm using qubit recycling', *Nature Photonics*, 6, pp. 773–776.

McEliece, R.J. (1978) 'A public-key cryptosystem based on algebraic coding theory', *DSN Progress Report*, 44, pp. 114–116.

Mosca, M. (2018) 'Cybersecurity in an era with quantum computers: will we be ready?', *IEEE Security & Privacy*, 16(5), pp. 38–41.

Nielsen, M.A. & Chuang, I.L. (2010) *Quantum Computation and Quantum Information*. 10th edn. Cambridge: Cambridge University Press.

NIST (2016) Post-Quantum Cryptography Standardization, National Institute of Standards and Technology. Available at: https://csrc.nist.gov/projects/post-quantum-cryptography (Accessed: date).

NIST (2022) 'Announcement of Round 3 of the NIST Post-Quantum Cryptography Standardization Process', *Federal Register*, 87(87).

Preskill, J. (2018) 'Quantum Computing in the NISQ era and beyond', *Quantum*, 2, 79.

Regev, O. (2006) ‘On lattices, learning with errors, random linear codes, and cryptography’, in S. Halevi (ed.) *Proceedings of the 37th Annual ACM Symposium on Theory of Computing (STOC ’05)*. New York: ACM, pp. 84–93.

Rivest, R.L., Shamir, A. & Adleman, L. (1978) ‘A method for obtaining digital signatures and public-key cryptosystems’, *Communications of the ACM*, 21(2), pp. 120–126.

Shor, P.W. (1994) ‘Algorithms for quantum computation: discrete logarithms and factoring’, in *Proceedings 35th Annual Symposium on Foundations of Computer Science*, Santa Fe, NM, 20–22 November 1994. Los Alamitos, CA: IEEE Computer Society, pp. 124–134.

APPENDIX A: FULL BENCHMARK DATA

Full Benchmark results can be found in source folder in allbenchmark.json by opening it as a text file.

APPENDIX B: FULL CODE LISTINGS

Full commented code can be found in the source folder.

APPENDIX C: ALL PERFORMANCE RESULTS

All performance results can be generated at each run of the program and are stored in a html. I have saved my own iteration in the source folder